IMovie – Requirement Analysis

Date: 25 October 2026 · **Team ID:** [add team ID] · **Project Name:** IMovie – Movie Ticket Booking Application

1. Customer Journey Map

Stage	User action	Touchpoint	Pain point today	How IMovie helps	Build status
Discover	Looks for a movie to watch	Home page, Movies page	Hard to know what is playing	Popular movies with poster and release date	Done
Choose show	Picks a theatre, date and time	Show list on the movie page	Showtimes scattered across places	Shows listed per movie with theatre, date, time	Planned
Select seats	Chooses tier and seats	Seat map	Unsure which seats are free	Seat map with balcony, middle and lower tiers and prices	Planned
Pay	Pays for the selected seats	Payment screen	Cash only or long queues	Online payment with verification	Planned
Confirmation	Receives proof of booking	Confirmation page and email	No instant proof	Booking id shown and emailed	Planned
After booking	Checks or cancels booking	My Bookings page	Calling the theatre	Self-service booking history	Planned

2. Solution Requirements

2.1 Functional Requirements

FR No.	Requirement	Sub-requirements	Status
FR-1	User registration	Register with username, email and password; Google sign-up	Not built (UI only)

FR No.	Requirement	Sub-requirements	Status
FR-2	User login	Email and password login, Google login, JWT session, logout	Not built (UI only)
FR-3	Movie browsing	Popular movies from TMDB; search and genre filter	Browsing done; search planned
FR-4	Theatre management (admin)	Add theatre with three seat tiers, tier prices, rows and seats per row	Done
FR-5	Show management (admin)	Add show with movie, theatre, date and time; list and edit shows	Form UI only
FR-6	Seat selection	Seat map generated from theatre layout; booked and held seats blocked	Not built
FR-7	Booking	Create booking, hold seats for a limited time, store booking record	Not built
FR-8	Payment	Create payment order, verify payment, mark booking confirmed	Not built
FR-9	Confirmation email	Send booking details by email after successful payment	Not built
FR-10	Booking history	User sees own bookings; admin sees bookings for own theatres	Placeholder page

2.2 Non-functional Requirements

NFR No.	Requirement	Description
NFR-1	Usability	Booking completed in a few clicks; responsive layout for mobile and desktop
NFR-2	Security	Hashed passwords, JWT authentication, role-based access for admin pages, secrets kept in environment variables, restricted CORS

NFR No.	Requirement	Description
NFR-3	Reliability	A seat can never be sold twice; failed payments release held seats; email failure must not cancel a confirmed booking
NFR-4	Performance	Pages load quickly and APIs respond fast under normal load (targets in the Performance Testing document)
NFR-5	Availability	The application should be reachable whenever the hosting service is up; database hosted on a managed service when deployed
NFR-6	Scalability	Stateless API so it can be scaled; indexes on show and booking queries

3. Data Flow Diagram

Level 0 (context):

```
[User] --browse, select seats, pay--> ( IMovie System ) --movies, seat map, confirmation--> [User]
[Admin] --theatres, shows--> ( IMovie System ) --bookings report--> [Admin]
( IMovie System ) <--movie data--> [TMDB API]
( IMovie System ) <--payment order and status--> [Payment Gateway]
( IMovie System ) --booking email--> [Email Service]
```

Level 1 processes and data stores:

Process	Input	Output	Data store
1. Authenticate	Credentials or Google token	JWT session	D1 Users
2. Browse movies	Request for movie list	Movie list	D2 Movies (TMDB and seeded data)
3. Manage theatres	Theatre details from admin	Saved theatre	D3 Theatres
4. Manage shows	Movie, theatre, date, time	Saved show	D4 Shows

Process	Input	Output	Data store
5. Select seats and book	Show id, chosen seats	Pending booking with held seats	D5 Bookings
6. Process payment	Booking id, payment details	Verified payment, confirmed booking	D5 Bookings
7. Send confirmation	Confirmed booking	Email to user	D1 Users, D5 Bookings

4. User Stories

User type	Story ID	I want to	So that	Acceptance criteria	Priority	Release
Customer	USN-1	Register with email and password	I can create an account	Account saved, password stored hashed	High	Sprint 2
Customer	USN-2	Log in and out	I can access my bookings securely	Valid login returns a session	High	Sprint 2
Customer	USN-3	Sign in with Google	I do not need another password	Google login creates or finds my account	Medium	Sprint 5
Customer	USN-4	Browse popular movies	I can decide what to watch	Posters and release dates shown	High	Sprint 1 (done)
Customer	USN-5	See shows for a movie	I can pick a theatre and time	Shows list theatre, date and time	High	Sprint 3
Customer	USN-6	Select seats from a seat map	I get the seats I want	Booked seats cannot be selected	High	Sprint 3
Customer	USN-7	Pay online	My booking is confirmed	Payment verified before confirmation	High	Sprint 4
Customer	USN-8	Receive a confirmation email	I have proof of booking	Email contains booking id and seat details	High	Sprint 4
Customer	USN-9	View my bookings	I can check my tickets	List shows past and upcoming bookings	Medium	Sprint 5
Admin	USN-10	Add a theatre with seat tiers and prices	Users can book there	Theatre saved with layout and prices	High	Sprint 1 (done)

User type	Story ID	I want to	So that	Acceptance criteria	Priority	Release
Admin	USN-11	Add shows for a movie	Users can book those shows	Show saved and visible to users	High	Sprint 2
Admin	USN-12	See bookings for my theatres	I can track sales	Bookings list on dashboard	Medium	Sprint 5

5. Technology Stack

Layer	Technology	Purpose	Status
Frontend	React 19, Vite 7	Single-page user interface	In use
Routing	React Router v7	Page navigation	In use
HTTP client	Axios	Calls to backend and TMDB	In use
Styling	Bootstrap (CDN) and custom CSS	Layout and look	In use
Backend	Node.js, Express 5, CORS	REST API	In use
Database	MongoDB with Mongoose 9	Users, theatres, shows, bookings	In use
External data	TMDB API	Movie catalogue	In use
Authentication	bcrypt, JSON Web Tokens	Password hashing and sessions	Planned
OAuth	Google OAuth 2.0	Social login	Planned
Payments	Razorpay (proposed)	Online payment	Planned
Email	Nodemailer with an SMTP provider (proposed)	Confirmation email	Planned
Tooling	ESLint, Git and GitHub	Code quality and version control	In use